Knowl Inf Syst (2015) 45:247–270
DOI 10.1007/s10115-014-0794-3

REGULAR PAPER

# Class imbalance revisited: a new experimental setup to assess the performance of treatment methods

**Ronaldo C. Prati · Gustavo E. A. P. A. Batista · Diego F. Silva**

Received: 26 September 2013 / Revised: 17 April 2014 / Accepted: 4 October 2014 /
Published online: 17 October 2014


**Abstract** In the last decade, class imbalance has attracted a huge amount of attention from researchers and practitioners. Class imbalance is ubiquitous in Machine Learning, Data Mining and Pattern Recognition applications; therefore, these research communities have responded to such interest with literally dozens of methods and techniques. Surprisingly, there are still many fundamental open-ended questions such as "Are all learning paradigms equally affected by class imbalance?", "What is the expected performance loss for different imbalance degrees?" and "How much of the performance losses can be recovered by the treatment methods?". In this paper, we propose a simple experimental design to assess the performance of class imbalance treatment methods. This experimental setup uses real data set with artificially modified class distributions to evaluate classifiers in a wide range of class imbalance. We apply such experimental design in a large-scale experimental evaluation with 22 data set and seven learning algorithms from different paradigms. We also propose a statistical procedure aimed to evaluate the relative degradation and recoveries, based on confidence intervals. This procedure allows a simple yet insightful visualization of the results, as well as provide the basis for drawing statistical conclusions. Our results indicate that the expected performance loss, as a percentage of the performance obtained with the balanced distribution, is quite modest (below 5 %) for the most balanced distributions up to 10 % of minority examples. However, the loss tends to increase quickly for higher degrees of class imbalance, reaching 20 % for 1 % of minority class examples. Support Vector Machine is the classifier paradigm that is less affected by class imbalance, being almost insensitive to all but the most imbalanced distributions. Finally, we show that the treatment methods only partially recover the performance losses. On average, typically, about 30 % or less of the performance that was lost due to class imbalance was recovered by these methods.

R. C. Prati (✉)
Centro de Matemática, Computação e Cognição, Universidade Federal do ABC,
Santo André, SP, Brazil
e-mail: ronaldo.prati@ufabc.edu.br

G. E. A. P. A. Batista · D. F. Silva
Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo,
São Carlos, Brazil

**Keywords** Class imbalance · Experimental setup · Sampling methods

## 1 Introduction

In the last decade, class imbalance has attracted a huge amount of attention from researchers and practitioners. Class imbalance is ubiquitous in Machine Learning, Data Mining and Pattern Recognition tasks, as it has been extensively documented in the literature with applications such as diagnostics of rare diseases [9], fraud detection [27], identification of oil spills in satellite radar images [23] and many others. Literally, hundreds of research papers have reported that imbalanced data lead to performance losses, and some sort of treatments—such as sampling [24], cost-sensitive learning [33,35], ensembles [16,32], among others—are able to improve classification.

Although the relationship between class imbalance and performance loss is well documented, we argue that it is under-comprehended. For instance, we have been inquired several times by different researchers about "from what distribution can a data set be considered imbalanced?". Although it is a quite naïve question, we were never able to fully answer it, since we are not aware of any definitive study relating performance loss, degree of class imbalance and learning systems.

Virtually, every paper about class imbalance has the exact same experimental setup. A proposed method is compared against one or two competing methods over a dozen or so data set. Although this experimental setup is reasonable to support an argument that the new method is as good as or better than the state of the art, it still leaves many unanswered questions. For sake of clarity, let us use an example: suppose that for a given application, a classifier over imbalanced data results in 80 % AUC, and after the application of a certain treatment method, we obtain 90 % AUC. It is indeed a significant improvement in classification; however, "were we able to fully recover the performances losses caused by class imbalance?".

We believe this question is extremely relevant and completely unanswered by the current research in class imbalance. The literature has provided dozens of methods to treat class imbalance, and there is (or will be) no single method able to provide the best performance for all data set. Therefore, given a problem in which we still need to improve the classification performance after the application of a treatment method $A$, we would like to know whether it is worth seeking for a different method $B$ to apply instead of $A$, hoping that $B$ will outperform $A$.

The reader should have anticipated that these two questions are directly related. If we knew the relationship between imbalance degree and performance loss, we could evaluate when a treatment method has recovered most of the loss. Unfortunately, no existing analysis is able to provide the answers we are looking for every pair of data set and learning system. However, we can answer these questions in terms of *expected* performance.

In this paper, we propose a simple experimental design to assess the performance of class imbalance treatment methods and classifiers. This experimental setup uses real data set with artificially modified class distributions to evaluate classifiers in a wide range of class imbalance. We divided the evaluation into two parts. The first part consists in inducing a classifier for each class distribution and measuring the performance loss compared to the balanced distribution. We found out that Support Vector Machine is the classifier paradigm that is the least affected by class imbalance, being insensitive to almost all but the most imbalanced distributions.

The second part consists of applying a treatment method and inducing a classifier for each class distribution. This time we measured the percentage of the performance loss that was recovered by the treatment method. In other words, 100 % represents that the classifier

induced after the application of the treatment method obtained the same performance of the classifier induced over (original) balanced data. We used two well-known over-sampling methods, random over-sampling and SMOTE, as well as two SMOTE variations, Borderline-SMOTE and ADASYN. We also used a MetaCost, a general cost-sensitive procedure that can be applied to any learning algorithm. We show that the *expected* performance recovery for all methods is typically about 30 % or less for the most imbalanced distributions. MetaCost featured negatively among the treatment methods, not being able to recover most of the losses.

To support the draw of statistical conclusions, we also propose a statistical procedure to evaluate the results. This procedure constructs confidence intervals about the performance loss relative to the balanced class distribution. A confidence interval is a range of values, calculated from the observed data, which is likely to contain the true value at a specified probability chosen by the researcher. Confidence intervals provide information that may be used to test hypotheses, as well as can be visualized in a graph for a simple although flexible and insightful visualization of the results.

We believe that papers that propose new treatment methods for class imbalance should adopt the proposed experimental setup. The proposed setup allows not just comparing a new method to competing methods, but also to compare the new method to a reference performance provided by the balanced data set.

This work is organized as follows: Sect. 2 describes the experimental design proposed in this work; Sect. 3 analyses how learning systems belonging to different paradigms perform under different class imbalance degrees; Sect. 4 analyses the ability of two sampling techniques to recover the performance lost due to class imbalance; Sect. 5 presents the proposed statistical procedure based on confidence intervals. Section 6 discusses possible limitations of our approach; Sect. 7 compares the proposed approach to other experimental setups in the literature; finally, Sect. 8 presents our conclusion and suggestions for future work.

## 2 Experimental design

Due to the lack of space, we assume the reader is familiar with the class imbalance problem and methods. In the case this assumption does not hold, the literature has several comprehensible surveys, and we recommend [20]. For the same reason, we are not able to include here all numerical results obtained in our experiments. Therefore, we decided to create a paper Web site [29] that has detailed results, including tables, data and scripts we have used to calculate and plot the confidence intervals; however, we note this paper is totally self-contained.

Our experimental design is inspired by the design used in [36]. The central idea is to generate several training set distributions with increasing degrees of class imbalance *while maintaining the training sets with a fixed number of examples*. Our training set distributions range from the balanced distribution (denoted as 50/50[1]) to the imbalanced distributions of 1/99 and 99/1. In order to generate these class distributions, we restricted the training sets to have a total number of instances equals to the number of instances of the minority class. We avoid using extremely imbalanced data set; in particular, the combination of a small data set with a large class imbalance would result in a training set with too few examples. We return to this discussion in Sect. 6, where we comment possible limitations of this work.

The test sets have the naturally occurring class distributions. We reserved 25 % of the original data as test set using a stratified sample and did not touch this portion of the data

[1] We use the notation $X/Y$, with $X + Y = 100$ to denote that for a set of 100 instances, $X$ belongs to the positive class and $Y$ belongs to the negative class.

until the final evaluation; the remaining 75 % was used as training set, with class distributions manipulated as previously described. In order to reduce variance and increase statistical significance, we repeated the whole process a hundred times using different train and test sample partitions.

For this specific study, we assembled a database with twenty-two data set. Since we want to promote reproducibility, most of them are public domain benchmark data set available in repositories such as UCI Machine Learning Repository [14] or used in projects like Statlog [26]. A few data set are not public domain, but we decided to include them since they are frequently used in studies involving class imbalance. These data set include tumor identification in mammography images [6,17]. We also included data set obtained in past research, and we make them publicly available for the first time in the paper Web site.

We use the area under the ROC curve (AUC) [12,28] as the main measure to assess our results. Since we chose AUC and also because we want to control the degree of class imbalance, we converted all non-binary-class data set into binary class, associating a positive label to one of the classes (usually the minority one) and aggregating all remaining classes into a negative class. Table 1 presents a summarized description of the data set included in our study. The table lists the data set full names, as well as a short identifier used in this paper, the number of instances and attributes, labels associated with the positive and negative classes and attribute class distribution. The data set are listed in increasing order of class imbalance.

Two data set resulted in two entries each in Table 1, because different classes were used as positive class. For the Letter data set, Letter-a is the variation in which the positive class is the original letter "a" class; and Letter-vowel has the positive class composed by all classes that represent vowels. For the Splice-junction Gene Sequences data set, one entry has the intron–exon ("ie") boundaries as positive class and the other has the exon–intron ("ei") boundaries. The final number of data set is 22, considering the four entries generated from these two data set.

We included at least one representative of each major learning paradigm. We selected C4.5 (decision trees), C4.5Rules (rules extracted from decision trees), CN2 and RIPPER (decision rules), Back-propagation Neural Network (connectionism), Naïve Bayes (Probabilistic) and Support Vector Machines (Statistical Learning). Whenever possible, we used the original implementations of the inducers that is the case for C4.5, C4.5Rules, CN2 and Ripper.

C4.5 and C4.5Rules are the original implementation provided by Quinlan [31]. We used the default parameters, with exception of pruning. Several research papers have pointed out that C4.5 pruning is hardly beneficial in imbalanced domains. Therefore, we induced unpruned trees. CN2 is the original implementation provided by Clark and Boswell [8]; we used it with default parameters. The same occurred to Ripper, which is the implementation provided by Cohen [10].

For Naïve Bayes and Neural Networks, we used the implementation provided by Borgelt [4]. In order to estimate conditional probabilities, Naïve Bayes uses a frequency table for symbolic attributes and normal distribution for continuous attributes. For Neural Networks, most domains required a simple configuration with no hidden layer, and the learning lasted for 1,000 epochs. For SVM, we used LIBSVM [5] with radial basis kernel with degree 3. For MetaCost, we drew 50 bootstrap samples, as suggested in [11].

We use this experimental design to answer the questions raised in abstract and introduction. We start discussing the expected performance loss for different learning paradigms in the next section.

**Table 1** Data set description

| Data set name | Identifier | #Examples | #Attributes (num., nom.) | Classes (min., maj.) | Classes % |
|---|---|---|---|---|---|
| Chess—King–Rook versus King–Pawn | Kr-versus-kp | 3,196 | 36 (0, 36) | (nowin, won) | (47.77, 52.23) |
| Contraceptive method choice | CMC | 1,472 | 9 (2,7) | (1, others) | (42.73, 57.27) |
| Liver disorders | Bupa | 345 | 6 (6, 0) | (1, 2) | (42.02, 57.98) |
| Tokyo SGI server performance data | Tokyo1 | 958 | 43 (43,0) | (good, bad) | (36.12, 63.88) |
| MAGIC gamma telescope data set | Magic | 19,019 | 10 (10,0) | (h, g) | (35.16, 64.84) |
| Breast cancer Wisconsin | Breast | 699 | 10 (10, 0) | (benign, malignant) | (34.99, 65.01) |
| Pima Indians diabetes | Pima | 768 | 8 (8, 0) | (1, 0) | (34.77, 65.23) |
| Tic–Tac–Toe endgame | Tic–Tac–Toe | 958 | 9 (0, 9) | (positive, negative) | (34.65, 65.35) |
| German credit data | German | 1,000 | 20 (7, 13) | (Bad, good) | (30.00, 70.00) |
| Splice-junction gene sequences—ie | Splice-ie | 3,176 | 60 (0, 60) | (ie, others) | (24.09, 75.91) |
| Splice-junction gene sequences—ei | Splice-ei | 3,176 | 60 (0, 60) | (ei, others) | (23.99, 76.01) |
| Vehicle silhouettes | Vehicle | 946 | 18 (18, 0) | (van, others) | (23.52, 76.48) |
| Letter recognition-vowel | Letter-vowel | 20,000 | 16 (16, 0) | (vowels, others) | (19.39, 80.61) |
| Pen-based recognition of handwritten digits | Pen-digits | 10,991 | 16 (16,0) | (2, others) | (10.41, 89.59) |
| Page blocks classification | Page-blocks | 5,472 | 10 (10,0) | (others, text) | (10.22, 89.78) |
| Optical recognition of handwritten digits | Opt-digits | 5,619 | 63 (63,00) | (2, others) | (9.91, 90.09) |
| Landsat satellite | Satimage | 6,435 | 36 (36, 0) | (4, others) | (9.73, 90.27) |
| Hoar-frost detection | Hoar-frost | 3,043 | 236 (200, 36) | (positive, negative) | (6.11, 93.9) |
| Letter recognition—A | Letter-a | 20,000 | 16 (16, 0) | (a, others) | (3.95, 96.05) |
| Abalone data set | Abalone | 4,176 | 8 (7,1) | (15, others) | (2.72, 97.28) |
| Nursery | Nursery | 12,960 | 8 (8, 0) | (not-recom, others) | (2.55, 97.45) |
| Microcalcifications in mammography | Mammography | 11,182 | 6 (6, 0) | (2, 1) | (2.32, 97.68) |

## 3 Class imbalance, performance loss and learning paradigms

The results are summarized in Table 2. Due to the lack of space, we only present the average results (with the corresponding standard deviation in the following line, between brackets) over all data set. The interested reader can find detailed results in the paper Web site [29]. Each reported value is a performance loss relative to the balanced distribution, as defined by Eq. 1:

**Table 2** Performance loss in AUC (percentage)

| Inducer | Class distribution (positive/negative) | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | 1/99 | 5/95 | 10/90 | 20/80 | 30/70 | 40/60 | 50/50 | 60/40 | 70/30 | 80/20 | 90/10 | 95/5 | 99/1 |
| C4.5 | 22.74 | 9.15 | 4.65 | 1.71 | 0.63 | 0.19 | 0.00 | 0.23 | 0.88 | 1.84 | 4.75 | 8.99 | 23.23 |
| | (9.69) | (5.75) | (3.94) | (1.67) | (0.97) | (0.74) | (0.00) | (0.32) | (1.14) | (1.73) | (3.37) | (5.58) | (11.56) |
| C4.5Rules | 24.12 | 10.98 | 6.55 | 2.84 | 1.18 | 0.35 | 0.00 | 0.48 | 1.45 | 2.90 | 6.71 | 11.01 | 25.08 |
| | (11.45) | (7.03) | (5.64) | (2.79) | (1.54) | (0.89) | (0.00) | (0.68) | (1.53) | (2.63) | (4.53) | (6.68) | (11.74) |
| CN2 | 19.79 | 6.50 | 3.15 | 1.01 | 0.22 | −0.03 | 0.00 | 0.37 | 0.83 | 1.50 | 3.44 | 6.54 | 16.40 |
| | (11.07) | (5.91) | (2.75) | (1.60) | (0.89) | (0.66) | (0.00) | (0.77) | (1.15) | (1.62) | (3.87) | (6.52) | (10.56) |
| Neural Network | 16.76 | 7.34 | 4.69 | 1.73 | 0.64 | 0.05 | 0.00 | 0.04 | 0.32 | 0.92 | 3.12 | 5.63 | 13.80 |
| | (9.51) | (6.45) | (5.32) | (2.02) | (0.87) | (0.53) | (0.00) | (0.51) | (1.25) | (1.98) | (4.83) | (6.69) | (9.79) |
| Naïve Bayes | 25.53 | 7.07 | 4.09 | 1.64 | 0.70 | 0.16 | 0.00 | 0.10 | 0.31 | 1.08 | 3.09 | 6.08 | 22.81 |
| | (14.75) | (9.32) | (7.78) | (3.86) | (1.31) | (0.60) | (0.00) | (0.59) | (2.00) | (4.16) | (7.85) | (9.40) | (15.42) |
| Ripper | 31.83 | 18.83 | 12.31 | 5.88 | 2.71 | 0.89 | 0.00 | 0.29 | 2.18 | 5.47 | 11.02 | 16.61 | 29.53 |
| | (8.45) | (8.04) | (5.75) | (4.06) | (2.57) | (1.11) | (0.00) | (2.23) | (3.28) | (5.50) | (7.72) | (9.25) | (8.07) |
| SVM | 11.39 | 0.42 | −3.19 | −5.50 | −4.72 | −2.87 | 0.00 | −2.93 | −3.18 | −3.16 | −2.18 | 0.78 | 11.77 |
| | (18.09) | (11.72) | (12.86) | (16.64) | (16.02) | (13.91) | (0.00) | (13.70) | (13.25) | (12.54) | (11.06) | (9.94) | (16.45) |
| Average | 21.74 | 8.61 | 4.61 | 1.33 | 0.19 | −0.18 | 0.00 | −0.20 | 0.40 | 1.51 | 4.28 | 7.95 | 20.37 |

$$L = \frac{B - I}{B} \tag{1}$$

where $B$ stands for the performance obtained with the balanced distribution and $I$ for the performance obtained with an imbalanced distribution. As previously described, $B$ and $I$ are measured in this work as the area under the ROC curve (AUC).

At this point, we can review the question "from what distribution can a data set be considered imbalanced?" Technically, the answer is every non-balanced distribution, since most learning systems (except SVM) show some degree of performance loss for every non-balanced distribution. Obviously, some practitioners might consider small losses insignificant; in that case, the distributions in the range 20/80–80/20 usually present small losses, on average bellow 2 %. Imbalanced distributions above and including 10/90 (and 90/10) tend to have more expressive losses, above 5 %; and the most imbalanced distributions in our study (1/99 and 99/1) had losses around 20 % on average. From the practical standpoint, we can answer the previous question by saying that for most learning systems the losses due to class imbalance start to be significant when the minority class represents 10 % of the data set or less.

We also posed the following question: "are all learning paradigms equally affected by class imbalance?" As we can see from the results in Table 2, the answer is clearly no. RIPPER is the learning algorithm that was the most affected by class imbalance. In contrast, SVM is very little affected by all but the most imbalanced distributions, obtaining even negative performance losses, i.e., small performance gains compared to the balanced distribution. A possible explanation is that SVM frequently uses few support vectors to determine the separation between classes, as previously observed by Japkowicz and Stephen [21]. However, SVM still suffer the consequences of severe class imbalance, as noted by Wu and Chang [37]. The authors conducted an experiment using two data set with class ratios of 10:1 and 10,000:1. In the second data set, the edge separation between classes tended to become more located over the space belonging to the minority class in comparison with the first data set. Therefore, the SVM inducer tended to classify more test cases as belonging to the majority class. In extreme situations, when the number of training examples from the minority class is not enough to characterize the decision space of this class, SVM might classify every instance as belonging to the majority class.

In the next section, we use the same experimental design to measure the capacity of treatment methods to recover the performance losses.

## 4 Class imbalance, performance recovery and treatment methods

So far we were able to characterize the expected performance losses for different class distributions and learning systems. These results lead to another question: "how much of the performance losses can be recovered by the treatment methods?" In this section, we use the same experimental design to answer this question.

A myriad of methods has been proposed to treat class imbalance. Unfortunately, due to the lack of space, we can only include the results of a few of them in this paper. We chose to analyze random over-sampling and SMOTE [6] for the following reasons: first, our previous experience with sampling methods [1] shows that these two methods outperform several other (over and under) sampling methods; second, SMOTE has became one of the most popular sampling methods for class imbalance, and it is frequently used as counterpart in empirical evaluations and has several proposed extensions and variations. We also included two of these variations: Borderline-SMOTE [18] and ADASYN [19]. Besides the sampling

methods, we also evaluated how cost-sensitive learning could help with class imbalances. We chose MetaCost [11], a general cost-sensitive procedure that can be applied to any classifier. This choice was motivated by the fact that not all learning algorithms used in our experiments have cost-sensitive implementations, neither is trivial to directly transform them to cope with costs.

Random over-sampling compensates the imbalanced class distribution by randomly replicating instances from the minority class. SMOTE, on the other hand, tries to accomplish this compensation by introducing synthetic examples. These synthetic examples are created by a linear interpolation between a minority class instance and its nearest neighbors. Borderline-SMOTE and ADASYN are two variations of SMOTE, which tries to focus on specific regions in the feature space.

Borderline-SMOTE calculates the $k$ nearest neighbors for each instance of the minority class. Let $m \leq k$ be the number of nearest neighbors belonging to the majority class. If $m = k$, the minority class example is considered noise and is discarded. If $m/2 \leq m < k$, the example of the minority class is marked as borderline. Borderline-SMOTE creates synthetic examples by interpolating borderline examples with their neighbors. Thus, the algorithm strengthens border regions, favoring the minority class.

ADASYN uses a density distribution as a criterion for deciding the number of synthetic examples generated from each minority class example. This factor is calculated according to the proportion of majority instances among the nearest neighbors of each minority example. ADASYN creates a larger number of synthetic examples for minority class examples that have higher factors, i.e., greater number of majority examples among the nearest neighbors. This strategy tends to create more minority class examples in borderline regions than in internal class regions.

MetaCost wraps a cost-minimization procedure around a learning algorithm. The idea is to draw bootstrap samples from the training set and to learn a (non-cost sensitive) classifier for each sample. For each instance in the training set, the predicted class probability for each class is averaged over all learned classifiers, and this probability is combined with a cost matrix to relabel the instances in the training set. The general idea is that examples with uncertain classification (for instance, with probability score near 0.5) are more likely to be relabeled according to the cost setting. The relabeled training set is then used to (indirectly) train a cost-sensitive classifier.

These treatment methods were applied in the same experimental setup used in the previous section. We saved all data partitions and applied the treatment methods in the exact same data used to measure the performance loss. The sampling methods were applied to all non-balanced data set, and new minority class examples were created until the training set became perfectly balanced. For MetaCost, the costs were defined so they also matched a perfectly balanced distribution. Once again, the test sets were not touched, and they keep the naturally occurring class distributions.

Our assessment measure is the performance recovery, measured as percentage of the performance loss defined by Eq. 1. The idea is to calculate the performance loss for the *treated* data set in the same way we did in the previous section for the *imbalanced* data. We include the equation here for sake of clarity:

$$L_{\mathrm{T}} = \frac{B - T}{B} \tag{2}$$

where $B$ stands for the performance obtained with the balanced distribution and $T$ for the performance obtained with the treated data, both measured in AUC.

In a second step, we calculate the performance recovery as a fraction of the performance loss for treated data over the performance loss for imbalanced data:

$$R = \frac{L - L_T}{L} \tag{3}$$

A simple way to understand Eq. 3 is to think that $R = 100\,\%$ when $L_T = 0\,\%$, or in other words, the performance recovery will reach 100 % when there is no performance loss for treated data. In that case, the classifier performance for treated data is the same as the performance for the balanced distribution. In contrast, $R = 0\,\%$ when $L_T = L$, or in other words, the treated data classifier had the same performance as the one induced with imbalanced data.

Although the semantics of the recovery measure is straightforward, its interpretation might be hindered in situations of small losses. Small loss values in the denominator of Eq. 3 may lead to high recovery rates due to spurious performance variations by chance. In other words, we can easily achieve a 1,000 % recovery rate if the treatment method obtains a marginal 1 % AUC improvement over a 0.1 % performance loss due class imbalance. In order to avoid large recovery rates due to small losses, we only analyze recovery rates for significant losses. The following tables show the recovery rates for combinations of data set, inducer and imbalance rate that had 10 % or greater performance loss in the experiment of Sect. 3. The interested reader can find all detailed results, including for loss rates smaller than such threshold in the paper Web site.

We believe this procedure leads to an more focused analysis of the results, avoiding distractions with spurious results. In general, we are interested in situations of higher performance losses, in which it is worth rebalancing the training data. Table 3 presents the expected performance recovery for random over-sampling.

Certainly, the most interesting values are the performance recovery associated with large class imbalance. For the class distributions of 1/99, 5/95 and 10/90 (and their negative class counterparts), we noticed that although some scattered number present recoveries above 50 %, most results are bellow 30 %. In our opinion, these numbers represent a rather modest performance recovery, especially if we consider that random over-sampling frequently provides competitive results with other sampling methods.

The results also show that not all learning systems are equally benefited by the replication of minority class examples. For example, the performance of Naïve Bayes has negative recovery rates for several class distributions. Apparently, the simple replication of examples does not help to correct the conditional probability estimates made by Naïve Bayes. These results contribute to the common criticism against the early class imbalance research that (inadvertently) considered fair to extrapolate to other inducers the results obtained with C4.5.

Table 4 presents the expected performance recovery for SMOTE. This method does not replicate examples from the minority class, but interpolates cases of this class in order to expand its decision space. A first observation about SMOTE is that it does not outperform random over-sampling by a large margin for any inducers. There is some improvements in performance in several configurations, although we can also notice that some configurations were the recovery of Random Oversampling is higher. We should note that SMOTE is considerably more computationally expensive than random over-sampling. Therefore, this performance gain obtained with some inducers comes with the cost of additional computational time.

Tables 5 and 6 present results for Borderline-SMOTE and ADASYN, two variations of SMOTE. Inspecting these tables, we can observe that there is no clear advantage of preferring them in spite of SMOTE. Again, there is some improvements in some configurations. How-

**Table 3** Performance recovery for Random Over-sampling in AUC (percentage)

| Inducer | Class distribution (positive/negative) | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | 1/99 | 5/95 | 10/90 | 20/80 | 30/70 | 40/60 | 50/50 | 60/40 | 70/30 | 80/20 | 0/10 | 95/5 | 99/1 |
| C4.5 | 19.18 | 63.08 | 64.93 | – | – | – | – | – | – | – | 67.74 | 57.20 | 28.96 |
| | (19.29) | (41.93) | (52.05) | – | – | – | – | – | – | – | (74.40) | (45.45) | (21.58) |
| C4.5Rules | 31.81 | 72.00 | 80.41 | – | – | – | – | – | – | 41.41 | 81.48 | 75.48 | 37.18 |
| | (22.72) | (39.28) | (41.39) | – | – | – | – | – | – | – | (26.83) | (29.49) | (26.33) |
| CN2 | 1.37 | 22.62 | – | – | – | – | – | – | – | – | 4.62 | 27.64 | 0.90 |
| | (19.54) | (26.56) | – | – | – | – | – | – | – | – | – | (29.82) | (17.27) |
| Neural Net | 7.24 | 40.90 | 66.47 | – | – | – | – | – | – | – | 63.45 | 44.94 | 0.42 |
| | (44.51) | (33.06) | (39.89) | – | – | – | – | – | – | – | (26.34) | (34.77) | (54.24) |
| Naïve Bayes | −1.51 | −14.95 | −8.65 | 12.68 | – | – | – | – | – | 11.00 | 3.34 | 2.18 | 2.07 |
| | (6.17) | (31.38) | (7.14) | – | – | – | – | – | – | – | (8.33) | (17.82) | (9.15) |
| Ripper | 7.90 | 21.84 | 39.08 | 60.03 | 52.66 | – | – | – | – | 126.71 | 113.15 | 92.62 | 56.64 |
| | (10.07) | (16.80) | (25.08) | (8.26) | – | – | – | – | – | (21.53) | (26.73) | (24.70) | (23.13) |
| SVM | 35.02 | 50.22 | 45.51 | – | – | – | – | – | – | – | 71.18 | 65.53 | 36.65 |
| | (28.09) | (23.57) | – | – | – | – | – | – | – | – | – | (15.00) | (39.43) |
| Average | 14.43 | 36.53 | 47.96 | 36.35 | 52.66 | – | – | – | – | 59.71 | 57.85 | 52.23 | 23.26 |

**Table 4** Performance recovery for SMOTE in AUC (percentage)

| Inducer | Class distribution (positive/negative) | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | 1/99 | 5/95 | 10/90 | 20/80 | 30/70 | 40/60 | 50/50 | 60/40 | 70/30 | 80/20 | 90/10 | 95/5 | 99/1 |
| C4.5 | 6.96 | 43.83 | 58.56 | – | – | – | – | – | – | – | 74.04 | 51.33 | 22.08 |
| | (28.12) | (44.03) | (68.00) | – | – | – | – | – | – | – | (107.33) | (34.34) | (32.09) |
| C4.5Rules | −3.47 | 32.79 | 58.02 | – | – | – | – | – | – | 79.07 | 45.51 | 43.05 | 23.54 |
| | (61.00) | (36.36) | (37.34) | – | – | – | – | – | – | – | (34.03) | (23.82) | (30.09) |
| CN2 | 7.89 | 29.08 | – | – | – | – | – | – | – | – | 5.69 | 36.14 | 5.19 |
| | (20.10) | (36.27) | – | – | – | – | – | – | – | – | – | (41.26) | (24.02) |
| Neural Net | −11.36 | 34.32 | 52.25 | – | – | – | – | – | – | – | 60.46 | 27.93 | −29.23 |
| | (68.02) | (52.42) | (74.78) | – | – | – | – | – | – | – | (42.17) | (53.32) | (88.10) |
| Naïve Bayes | −11.13 | −45.07 | −16.43 | −40.63 | – | – | – | – | – | −21.83 | −21.00 | −32.60 | −10.28 |
| | (21.41) | (46.93) | (11.08) | – | – | – | – | – | – | – | (15.58) | (61.67) | (18.92) |
| Ripper | 11.78 | 31.78 | 37.25 | 81.74 | 73.56 | – | – | – | – | 96.37 | 82.18 | 74.98 | 45.40 |
| | (19.88) | (30.18) | (35.53) | (20.29) | – | – | – | – | – | (46.55) | (27.20) | (22.78) | (26.02) |
| SVM | 24.27 | 22.23 | −6.66 | – | – | – | – | – | – | – | −1.70 | 44.71 | 11.89 |
| | (47.08) | (40.09) | – | – | – | – | – | – | – | – | – | (43.28) | (55.96) |
| Average | 3.56 | 21.28 | 30.50 | 20.56 | 73.56 | – | – | – | – | 51.20 | 35.03 | 35.08 | 9.80 |

**Table 5** Performance recovery for Borderline-SMOTE in AUC (percentage)

| Inducer | Class distribution (positive/negative) | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | 1/99 | 5/95 | 10/90 | 20/80 | 30/70 | 40/60 | 50/50 | 60/40 | 70/30 | 80/20 | 90/10 | 95/5 | 99/1 |
| C4.5 | 6.36 | 44.77 | 69.07 | – | – | – | – | – | – | – | 74.74 | 53.53 | 17.57 |
| | (27.35) | (43.59) | (63.08) | – | – | – | – | – | – | – | (106.34) | (32.38) | (32.38) |
| C4.5Rules | −2.63 | 39.56 | 55.87 | – | – | – | – | – | – | 69.11 | 52.49 | 41.85 | 16.46 |
| | (56.37) | (37.89) | (38.45) | – | – | – | – | – | – | – | (32.05) | (25.08) | (37.2)6 |
| CN2 | 7.39 | 30.14 | – | – | – | – | – | – | – | – | 7.81 | 32.86 | 4.48 |
| | (19.59) | (42.85) | – | – | – | – | – | – | – | – | – | (37.51) | (25.29) |
| Neural Net | −5.53 | 29.08 | 0.48 | – | – | – | – | – | – | – | 54.13 | 20.77 | −30.08 |
| | (63.86) | (43.71) | (49.08) | – | – | – | – | – | – | – | (27.07) | (52.88) | (80.68) |
| Naïve Bayes | −11.30 | −43.44 | −16.38 | −34.75 | – | – | – | – | – | −20.06 | −21.00 | −32.49 | −10.16 |
| | (21.79) | (46.77) | (11.02) | – | – | – | – | – | – | – | (15.58) | (61.44) | (18.50) |
| Ripper | 11.37 | 31.53 | 45.06 | 65.04 | 67.87 | – | – | – | – | 95.93 | 78.86 | 76.80 | 43.08 |
| | (18.58) | (29.49) | (34.30) | (11.69) | – | – | – | – | – | (18.24) | (24.90) | (22.78) | (25.38) |
| SVM | 26.61 | 22.61 | −0.24 | – | – | – | – | – | – | – | −1.70 | 44.71 | 13.43 |
| | (42.63) | (39.67) | – | – | – | – | – | – | – | – | – | (43.28) | (53.67) |
| Average | 4.61 | 22.04 | 25.64 | 15.15 | 67.87 | – | – | – | – | 48.33 | 35.05 | 34.01 | 7.83 |

**Table 6** Performance recovery for ADASYN in AUC (percentage)

| Inducer | Class distribution (positive/negative) | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | 1/99 | 5/95 | 10/90 | 20/80 | 30/70 | 40/60 | 50/50 | 60/40 | 70/30 | 80/20 | 90/10 | 95/5 | 99/1 |
| C4.5 | 6.23 | 43.27 | 76.02 | – | – | – | – | – | – | – | 63.60 | 51.61 | 16.62 |
| | (28.85) | (41.21) | (82.64) | – | – | – | – | – | – | – | (114.10) | (33.42) | (33.69) |
| C4.5Rules | −6.60 | 29.73 | 67.62 | – | – | – | – | – | – | 64.08 | 45.87 | 44.69 | 19.16 |
| | (60.03) | (37.39) | (38.23) | – | – | – | – | – | – | – | (36.85) | (24.71) | (29.10) |
| CN2 | 8.42 | 33.68 | – | – | – | – | – | – | – | – | 14.32 | 33.17 | 7.06 |
| | (20.05) | (49.46) | – | – | – | – | – | – | – | – | – | (39.67) | (24.55) |
| Neural net | −11.23 | 30.61 | 20.28 | – | – | – | – | – | – | – | 49.11 | 22.69 | −32.58 |
| | (67.08) | (46.78) | (81.02) | – | – | – | – | – | – | – | (40.20) | (50.28) | (91.38) |
| Naïve Bayes | −11.16 | −45.53 | −15.48 | −37.79 | – | – | – | – | – | −20.14 | −20.84 | −31.28 | −12.75 |
| | (21.58) | (48.19) | (9.69) | – | – | – | – | – | – | – | (15.81) | (60.43) | (23.15) |
| Ripper | 10.62 | 28.18 | 39.98 | 72.74 | 48.54 | – | – | – | – | 90.44 | 76.24 | 69.21 | 45.11 |
| | (20.91) | (30.94) | (35.95) | (10.52) | – | – | – | – | – | (27.94) | (25.66) | (22.72) | (27.34) |
| SVM | 23.93 | 22.37 | −4.36 | – | – | – | – | – | – | – | −3.25 | 44.66 | 13.20 |
| | (47.21) | (40.05) | – | – | – | – | – | – | – | – | – | (43.17) | (53.50) |
| Average | 2.89 | 20.33 | 30.68 | 17.47 | 48.54 | – | – | – | – | 44.79 | 32.15 | 33.54 | 7.97 |

ever, we can also notice that some configurations were SMOTE or random over-sampling leads to a higher recovery.

Table 7 presents results for MetaCost. Overall, the performance of MetaCost is quite poor, when compared to the sampling methods. A possible explanation is that, as MetaCost internally uses the prediction of the classifiers induced over imbalanced data, the costs were not able to compensate from the imbalance.

In general, we can say that the performance recoveries for both methods were rather modest. Although some scattered results present gains above 50 %, most of the performance recoveries are around or bellow 30 %.

## 5 Using confidence intervals to visualize results and draw statistical conclusions

The use of performance loss and performance recovery enables the construction of confidence intervals in a consistent way over data set, treatment methods and learning paradigms. Confidence interval provides a range of values within which the population parameter is likely to lie. The general expression of confidence intervals is $\rho \pm \epsilon$, where $\rho$ is the estimated parameter and $\epsilon$ defines a range of values (interval) that act as good estimates of the unknown population parameter as a function of the degree of uncertainty the users assumes for the parameter.

It has been argued recently that confidence intervals are a better approach to statistically analyze classification performance [3,13] than reporting $p$ values or the results of null hypothesis significance testing, due to the ability to interpret the confidence interval in a proper way, depending the effects we would like to analyze. These confidence intervals also allow the visualizations of the results in a graph, helping to draw statistical conclusions about the questions posed in this paper.

To this end, we adopted the `pairedCI` [15] package from the CRAN repository[2] to compute the confidence intervals. This package contains functions that can be used to construct parametric and nonparametric confidence intervals for ratios of values. In this paper, we used the nonparametric version [2] to construct 95 % confidence intervals for performance losses of original data and their counterpart treated by the class imbalance treatment methods.

The confidence intervals were constructed in three different ways: first, with performance loss for each class distribution over all data set and learning algorithms (Fig. 1); second, with the performance loss for each learning algorithm, over all data set and class distributions (Fig. 2); and third, with performance loss for each combination of learning algorithm and class distribution, over all data set (Fig. 3).

The confidence interval represents values for the population parameter for which the difference between the parameter and the observed estimate is not statistically significant at the $\alpha$% level, so it can be used to infer statistical significance at that level. In our setting, if the confidence interval for the performance loss does not include 0 %, then the performance of the classifier/treatment method significantly differs from the balanced distribution. Furthermore, when comparing two classifiers/treatment methods, if the confidence intervals of them do not overlap, then there is significantly statistical difference between the methods.

To gain some insight into the question about the performance loss for different imbalance degrees, Fig. 1 shows the 95 % confidence intervals for the average performance loss for each class distribution, averaged over all learning algorithms. The confidence interval for the

[2] CRAN (http://cran.r-project.org) is a network of Web servers distributed around the world that store versions of code and documentation for the statistical software R, as well as community contributed packages.

**Table 7** Performance recovery for MetaCost in AUC (percentage)

| Inducer | Class distribution (positive/negative) | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | 1/99 | 5/95 | 10/90 | 20/80 | 30/70 | 40/60 | 50/50 | 60/40 | 70/30 | 80/20 | 90/10 | 95/5 | 99/1 |
| C4.5 | −11.53 | 17.37 | 26.63 | – | – | – | – | – | – | – | 31.81 | 22.38 | −3.15 |
| | (28.22) | (31.96) | (28.24) | – | – | – | – | – | – | – | (3.82) | (22.83) | (19.06) |
| C4.5rules | −27.42 | 24.12 | 21.53 | – | – | – | – | – | – | 13.36 | 34.64 | 28.33 | −2.78 |
| | (62.14) | (30.52) | (36.43) | – | – | – | – | – | – | – | (26.93) | (29.45) | (17.09) |
| CN2 | −22.27 | 7.74 | – | – | – | – | – | – | – | – | −42.69 | −22.86 | −30.93 |
| | (28.57) | (15.54) | – | – | – | – | – | – | – | – | – | (37.86) | (37.04) |
| Neural Net | −76.58 | 9.44 | −125.80 | – | – | – | – | – | – | – | −38.81 | −22.25 | −71.04 |
| | (106.93) | (54.40) | – | – | – | – | – | – | – | – | (79.96) | (13.31) | (79.93) |
| Naïve Bayes | 12.06 | −42.16 | −4.62 | 15.90 | – | – | – | – | – | 16.76 | 7.02 | 28.59 | 15.94 |
| | (23.76) | (94.31) | (5.43) | – | – | – | – | – | – | – | (8.77) | (66.43) | (31.20) |
| Ripper | −18.65 | −11.12 | −4.78 | 22.45 | −71.72 | – | – | – | – | 50.13 | −10.99 | −9.57 | −17.09 |
| | (33.40) | (30.53) | (57.93) | (45.83) | – | – | – | – | – | (19.53) | (17.84) | (37.17) | (31.93) |
| SVM | 23.14 | 44.34 | 52.86 | – | – | – | – | – | – | – | 86.35 | 72.33 | −0.79 |
| | (31.55) | (23.60) | – | – | – | – | – | – | – | – | – | (37.36) | (84.63) |
| Average | −17.32 | 7.10 | −7.36 | 19.18 | −71.72 | – | – | – | – | 26.75 | 9.62 | 13.85 | −15.69 |

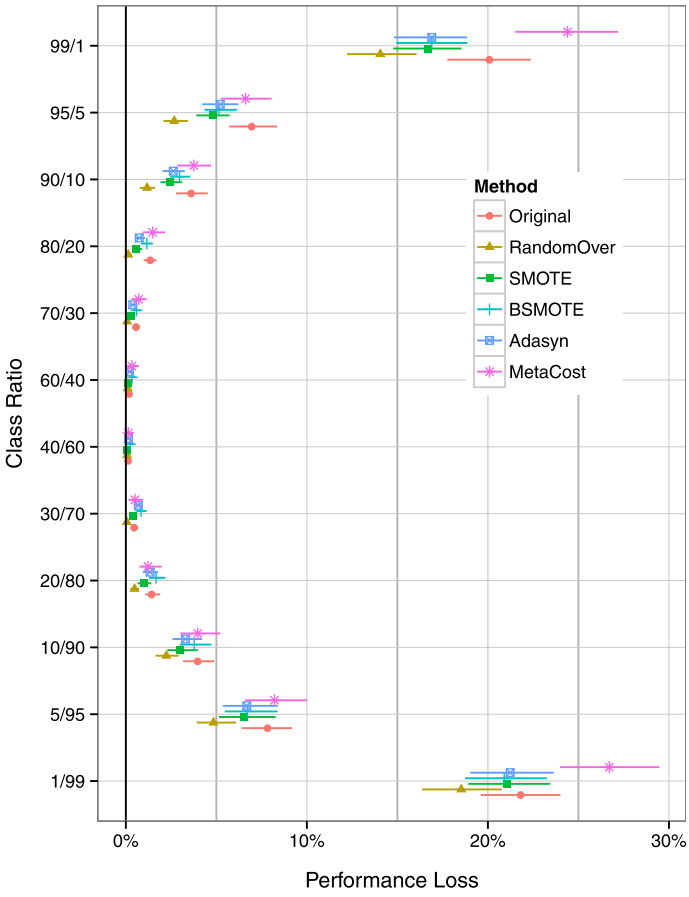


**Fig. 1** Confidence intervals for average performance loss and recovery for each class distribution

original (non-treated data) is shown in red marked with a dot, random over-sampling in brawn marked with a triangle, SMOTE in green marked with a filled square, Borderline SMOTE in cyan marked with a plus signal, Adasyn in blue marked with a square, MetaCost in pink marked with an asterisk. Interestingly, the performance of learning algorithms forms an "U" shaped pattern (flipped 90° clockwise), with performance loss closer to 0 % near balanced distributions, going upward the more unbalanced the class distribution is.

Also notice that this pattern occurs for both non-treated and treated data, although the non-treated data (original) diverges more accentuated from the performance from the balanced distribution than the treated data in all but MetaCost case. MetaCost had a poor performance for the most imbalanced configurations. This can be explained by the fact that MetaCost uses the classifiers' outputs from the bootstrap samples, and the pre-defined costs were not able to compensate from the losses for the most severe class imbalanced distributions. Indeed, in several cases, a trivial (majority class) classifier was induced in these extremes, increasing the performance loss rather than diminishing it. This is a known side effect of imbalanced data set in cost-sensitive classification [25].

As discussed in the previous section, this suggests three conclusions. First, class imbalance does harm classification performance, as the performance loss increases whenever the class

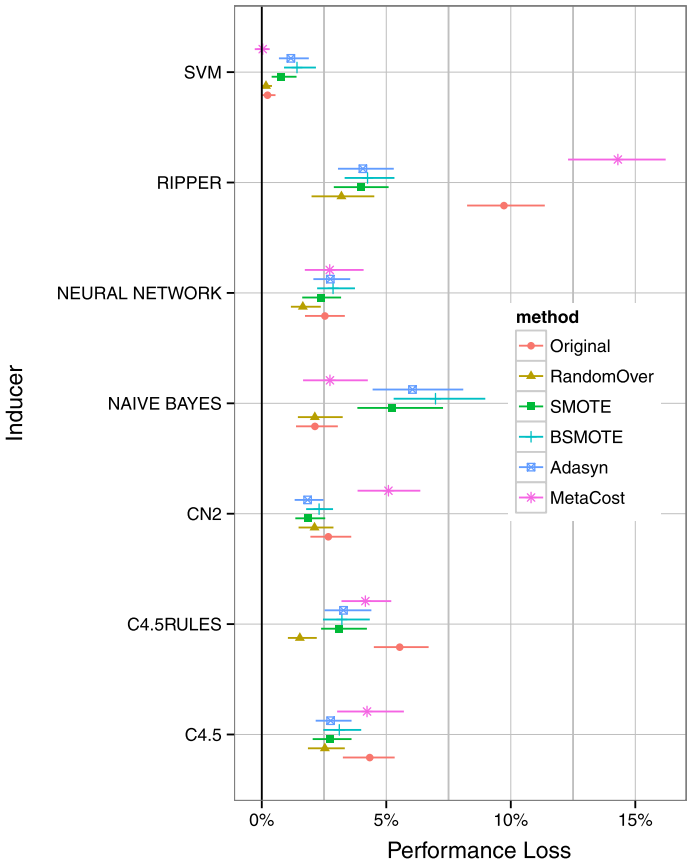


**Fig. 2** Confidence intervals for average performance loss and recovery for each learning algorithm

distribution diverges from the balanced one. Second, the treatment methods do contribute to alleviate the problem, although they are not able to fully recover the performance from the original balanced distribution. It is important to notice that even though the recovery is not close to the maximum, they are significant comparing to the original distribution for the most unbalanced distributions (in all but one case where the class distribution is equal or lower than 10/90 or 90/10 the confidence intervals do not overlap, indicating a significant result at 95 % confidence level). Third, it is difficult to use cost-sensitive learning, particularly MetaCost due to the influence of class imbalance in the cost reweighing process.

As a final observation, the performance of random over-sampling is surprisingly good compared to more sophisticated methods. For several class distributions, including the most imbalanced ones, it outperforms several competing methods with statistical significance.

We now turn to the question if all the learning algorithms are equally affected by class imbalance. Figure 2 shows 95 % confidence intervals for the average performance losses for each learning algorithm averaged over all the class distributions. We use the same color and shape combination as in the previous figure. From this picture, we can see that the most affected learning algorithms are Ripper and C4.5Rules, with average performance losses around 10 and 5 %, respectively. In contrast, these methods are also the most benefitted

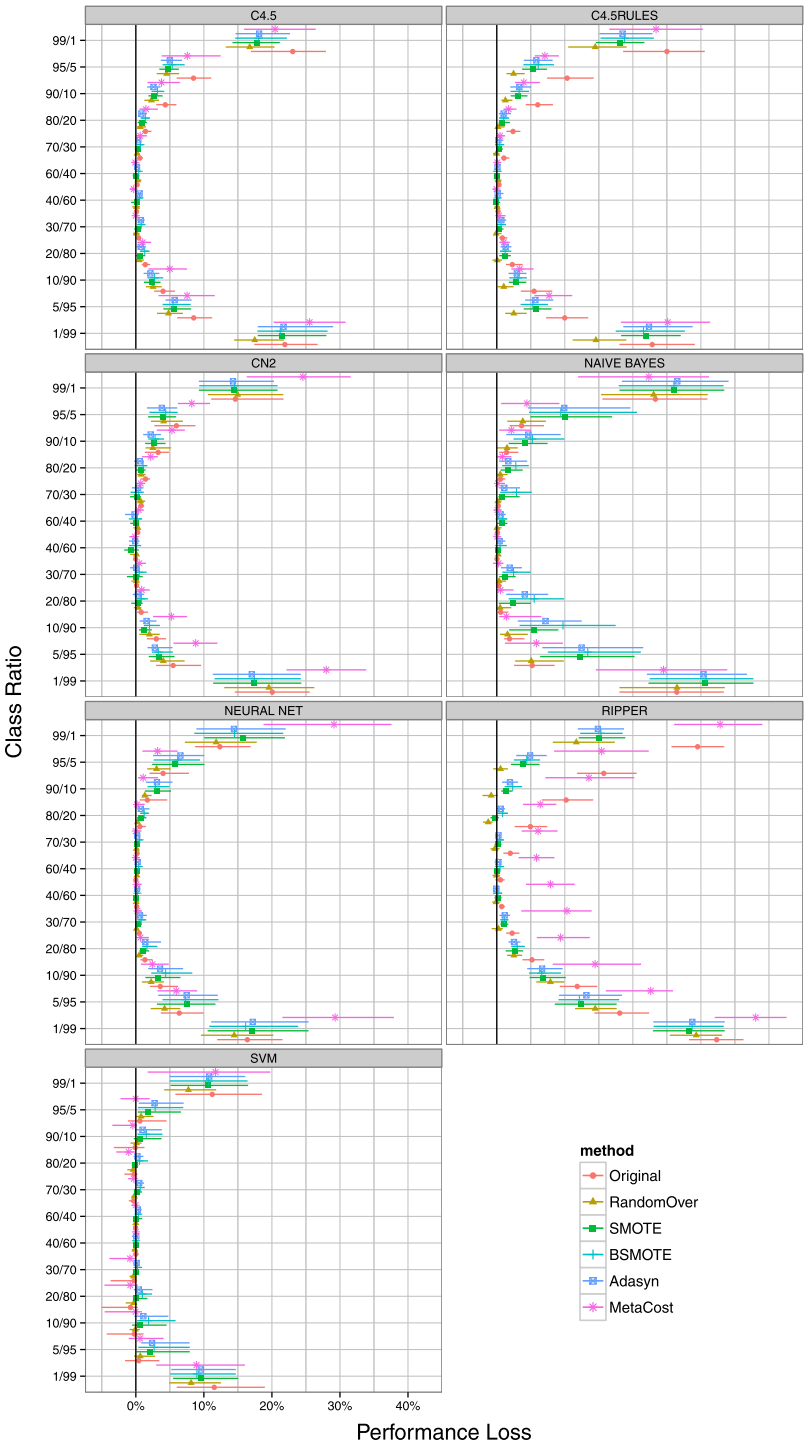


**Fig. 3** Confidence intervals for average performance loss and recovery for each learning algorithm

by the sampling treatment. The sampling methods produce results statistically significant when compared to the original non-treated data, as the confidence intervals do not overlap. MetaCost had a very poor performance with Ripper, statistically worst than the non-treated distribution. The treatment methods also seem to be good for C4.5, although there is no significant differences. Interestingly, the SMOTE variations seem to harm the performance of SVMs and Naïve Bayes.

Although Figs. 1 and 2 provide insightful information about these two questions, they may conclave some details as they average over all learning algorithms and class distributions, respectively. Figure 3 is similar to Fig. 1, but now there is a graph for each inducer separately. This figure shows the 95 % confidence intervals for the average performance loss for each class distribution and for each learning algorithms, averaged over all data set. We use the same color and shape combination as in the previous figures.

Even though the overall trend similar for all inducers (the performance of learning algorithms forms an "U" shaped pattern (flipped 90° clockwise), with performance loss closer to 0 % near balanced distributions, going upward the more unbalanced the class distribution is.), there are some interesting particularities. For the original (non-treated) distributions, the divergence from the balanced distribution is much more accentuated for Ripper, followed by C4.5Rules and C4.5. For these three inducers, there are significant differences for the distributions above 30/70 and 70/30. Naïve Bayes, CN2 and the Neural Network have a flatten curve, with a significant loss only for the distributions above 10/90 and 90/10. SVM, on the other hand, only has significant differences for the distributions 1/99 and 99/1. There are indeed distributions which seems to be better than the balanced one (an "negative" performance loss in the graph).

The treatment methods are beneficial for Ripper and C4.5Rules for almost all class distributions (with a few exceptions for C.45Rules) although, as discussed before, the recovery is only partial. C4.5 also presents some benefit, although it is not possible to indicate significative differences. SMOTE seems not to be beneficial for Naïve Bayes for very skewed distributions (1/99 and 99/1), although it is not possible to detect significative differences. For CN2, Neural Net and SVM, the few improvements are not significant, and in some cases worse than the original (without significant differences). MetaCost performed quite poorly for Ripper, CN2 and the Neural Network. Overall, there is *very* little difference for the SMOTE and its variations (Borderline-SMOTE and Adasyn).

A possible explanation for the differences in performance by the different learning algorithms are the way they explore the search space and the mechanism they adopt aiming to avoid overfitting. The most affected learning algorithms, Ripper and C4.5Rules, adopt rule pruning mechanisms in order to improve accuracy and reduce the rule set size. To this end, they use a separated "pruning set" taken from the training data set. Although this strategy may improve the overall accuracy (in the training set), this improvement could be rather artificial as it can be achieved by predicting the most frequent class in detriment of the less frequent, a phenomena well known in the literature [30]. On the other hand, as stated before, the less affected algorithm, SVM, requires only the properly definition of the support vectors in order to build the models. Only the most severe imbalanced class distributions would have a strong influence in this definition.

## 6 Limitations

We reserve this section to discuss some limitations of the proposed experimental setup. The most obvious limitation is that this experimental setup is restricted to binary-class problems.

Although it is possible to extend the approach to more than two classes, the number of results to be analyzed would increase exponentially. In contrast, binary-class problems are quite common in imbalanced domains, in which frequently the positive class is assigned to a class of interest and the negative class is associated to all remaining objects.

A less obvious limitation is the size of the training sets. Since we generated several distributions, from the positive class being a 1 % minority class up to the positive class being a 99 % majority class, we had to restrict the size of the training set as the size of the minority class. In order to partially overcome this limitation, we looked for more balanced data set or imbalanced data set with larger number of instances. However, this limitation prevented us to use in our experimental evaluation some imbalanced data set frequently present in imbalanced data papers.

A possible criticism is that the restricted sizes of the training set might be biasing the results in some way. For instance, some inducers that better deal with smaller data set might be favored. However, notice that we do not report any *absolute* performance results; all results are relative to the performance obtained with the balanced distribution. In addition, the training sets have a constant number of instances for all class distributions.

We should note, however, that the restricted size of the training sets may lead to very few minority class examples for the most imbalanced distributions. It is not uncommon to find data set in which the minority class is represented by a dozen or so examples under these class distributions. We must note that this is not a very uncommon situation, and it known as absolute rarity [34]. The literature has several examples of application domains that had class imbalance of the order of 1:1000, 1:10,000, or superior. In these domains, the learning systems almost invariantly have to deal with similar absolute number of minority class examples.

Finally, it is important to keep in mind that the results presented in this paper are averaged out over a series of data set. This is because the main point we want to highlight with our proposed methodology is a way to evaluate general trends about learning algorithms and treatment methods. Obviously, results can be very different for particular data set. The proposed method is quite general though and can used for a single data set as well. The paper Web site contains detailed results, including confidence interval plots for each data set.

## 7 Related work

The literature on class imbalance has several experimental papers with large-scale comparisons of learning algorithms or treatment methods. In this section, we compare the experimental setups of these papers to the one used here.

As we previously observed, our experimental design is inspired by the setup used in [36]. Weiss and Provost varied the training set class distribution in order to identify which distribution should be used when a limited number of examples is available. They conclude that, when AUC measure is used, although no single class distribution is able to provide the best-performing classifier, the balanced distribution performs well.

We used the results of Weiss and Provost to elect the balanced distribution as the reference distribution in our experiments. Although the results in [36] were obtained with the C4.5 decision tree inducer only, our results conform with their results. As the reader can observe in Table 2, the average results for all inducers present positive losses for all non-balanced distributions. The exception is SVM, in which the best distribution was 20/80 with an average improvement of 4.21 % over the balanced distribution. These are average results over all data

sets; we invite the interested reader to check the paper Web site for detailed (per data set) results.

An interesting counterpart to the experimental design of this paper is present in [7]. In that work, Cieslak and Chawla investigate classifier performance when testing class distributions change substantially. For instance, a disease outbreak may change the class prior probability considerably, making it differ substantially from the class distribution used in training. Therefore, in their experimental setup, a classifier is compared against several test sets with different class distributions.

Another experimental work is presented in [22]. Khoshgoftaar et al. are interested in studying classification performance when one class is rare. In their experimental setup, the positive class is represented by 5, 10, 20 or 40 instances, and the negative class is set up in such a way that the positive class represents from 65 to 1 % of the total number of examples. For instance, when there are 5 positive examples, the negative examples can be as low as 3 examples (65 % positive class) up to 495 (1 % positive class). Khoshgoftaar et al. conclude that the balanced distribution is outperformed by the distributions 2:1 (negative:positive) for 10, 20 and 40 positive examples, and by the distribution 3:1 for 5 positive examples.

An interesting feature of the experimental design used in [22] is to allow the analysis of classification performance factorized by class rarity and class distribution. Our experimental design does not allow the same analysis since, for instance, a 1 % class distribution may represent different absolute number of positive examples in different data set. In contrast, our experimental design leads to training sets with fixed number of examples, and therefore, the results are not influenced by the training sets sizes. The Khoshgoftaar et al. design cannot completely factor out the influence of training set sizes. This influence might even help to explain why the most rare configuration (with 5 positive examples) required a 3:1 class distribution when the other less rare distributions required only 2:1. We note that in such design, the most balanced distributions also have smaller training sets sizes.

All these papers, however, report results in terms of absolute performance values. As we argued in the introduction, this approach cannot answer relevant question about the class imbalance problem.

## 8 Conclusion and future work

This paper proposes an experimental design to evaluate the influence of class imbalance in classifiers performance. This experimental design allows to evaluate the performance loss caused by different degrees of class imbalance, as well as to measure the performance recovery obtained by treatment methods. We also proposed a statistical procedure based on confidence intervals to help draw statistical conclusions about the experiments.

We conducted an experiment to answer some open-ended questions about class imbalance and concluded that all evaluated classifiers are affected by the class imbalance problem. All classifiers, except SVM, present some loss of performance for all class distributions. This loss tends to be more expressive as the classes become more imbalanced. Additionally, SVM seems to be only susceptible to absolute rarity. For SVM, performance loss occurred for the proportions 1/99 and 99/1. In these proportions, the data set have only 1 % minority class cases and, considering the restricted size of the training sets, the absolute number of minority class examples is very limited, characterizing absolute rarity.

We also made experiments with treated data by sampling methods and a cost-sensitive approach. We measured how much class imbalance performance loss was recovered by artificially rebalancing the data. The over-sampling methods evaluated were able to occasionally

recover a significant proportion (between 50 and 60 %) of the performance lost. However, for the majority of the executions, the performance recovery was below 30 %, which can be considered a quite modest recovery rate.

As future work, we consider to evaluate additional sampling methods as well as other approaches to treat imbalanced data, such as other sampling algorithms, ensembles and cost-sensitive learning methods.

**Acknowledgments** We thank the anonymous reviewers for their comments on the draft of this paper. We also thank Nitesh Chawla for providing the Microcalcifications in Mammography data set. This work was funded by FAPESP award 2012/07295-3.

## References

1. Batista GEAPA, Prati RC, Monard MC (2004) A study of the behavior of several methods for balancing machine learning training data. SIGKDD Explor 6(1):20–29
2. Bennett BM (1965) Confidence limits for a ratio using Wilcoxon's signed rank test. Biometics 21(1):231–234
3. Berrar D, Lozano JA (2013) Significance tests or confidence intervals: which are preferable for the comparison of classifiers?. J Exp Theor Artif Intell 25(2):189–206. http://www.ingentaconnect.com/content/tandf/teta/2013/00000025/00000002/art00003
4. Borgelt C (2012) Christian borgelt web page. http://www.borgelt.net/
5. Chang C-C, Lin C-J (2012) Libsvm—a library for support vector machines. http://www.csie.ntu.edu.tw/cjlin/libsvm/
6. Chawla NV, Bowyer KW, Hall LO, Kegelmeyer WP (2002) SMOTE: synthetic minority over-sampling technique. J Artif Intell Res 16:321–357
7. Cieslak D, Chawla N (2008) Analyzing pets on imbalanced datasets when training and testing class distributions differ. In: Pacific-Asia conference on advances in knowledge discovery and data mining, pp 519–526
8. Clark P, Boswell R (1991) Rule induction with CN2: some recent improvements. In: European working session on machine learning, pp 151–163
9. Cohen G, Hilario M, Sax H, Hugonnet S, Geissbhler A (2006) Learning from imbalanced data in surveillance of nosocomial infection. Artif Intell Med 37(1):7–18
10. Cohen WW (1995) Fast effective rule induction. In: International conference on machine learning. Morgan Kaufmann, Los Altos, CA, pp 115–123
11. Domingos P (1999) Metacost: a general method for making classifiers cost-sensitive. In: Fayyad UM, Chaudhuri S, Madigan D (eds) ACM SIGKDD international conference on knowledge discovery and data mining. ACM, pp 155–164
12. Fawcett T (2006) An introduction to ROC analysis. Pattern Recognit Lett 27(8):861–874
13. Foody GM (2009) Classification accuracy comparison: Hypothesis tests and the use of confidence intervals in evaluations of difference, equivalence and non-inferiority. Remote Sens Environ 113(8):1658–1663. http://www.sciencedirect.com/science/article/pii/S0034425709000923
14. Frank A, Asuncion A (2010) UCI machine learning repository. http://archive.ics.uci.edu/ml
15. Froemke C, Hothorn L, Schneider M (2012) Confidence intervals for the ratio of locations and for the ratio of scales of two paired samples. Technical report, The Comprehensive R Archive Network. http://cran.r-project.org/web/packages/pairedCI/index.html
16. Galar M, Fernandez A, Barrenechea E, Bustince H, Herrera F (2012) A review on ensembles for the class imbalance problem: bagging-, boosting-, and hybrid-based approaches. IEEE Trans Syst Man Cybern Part C 42(4):463–484
17. Guo H, Viktor HL (2004) Learning from imbalanced data sets with boosting and data generation: the databoost-im approach. SIGKDD Explor 6(1):30–39
18. Han H, Wang W-Y, Mao B-H (2005) Borderline-smote: a new over-sampling method in imbalanced data sets learning. In: International conference on advances in intelligent computing. Lecture notes in computer science. Springer, Berlin, pp 878–887. doi:10.1007/11538059_91
19. He H, Bai Y, Garcia E, Li S (2008) Adasyn: adaptive synthetic sampling approach for imbalanced learning. In: IEEE international joint conference on neural networks, pp 1322–1328
20. He H, Garcia EA (2009) Learning from imbalanced data. IEEE Trans Knowl Data Eng 21(9):1263–1284

21. Japkowicz N, Stephen S (2002) The class imbalance problem: a systematic study. Intell Data Anal 6(5):429–449
22. Khoshgoftaar TM, Seiffert C, Hulse JV, Napolitano A, Folleco A (2007) Learning with limited minority class data. In: International conference on machine learning and applications, pp 348–353
23. Kubat M, Holte RC, Matwin S (1998) Machine learning for the detection of oil spills in satellite radar images. Mach Learn 30(2–3):195–215
24. Liu X-Y, Wu J, Zhou Z-H (2006) Exploratory under-sampling for class-imbalance learning. In: IEEE international conference on data mining, pp 965–969
25. Liu X-Y, Zhou Z-H (2006) The influence of class imbalance on cost-sensitive learning: an empirical study. In: 'ICDM', IEEE Computer Society, pp 970–974
26. Michie D, Spiegelhalter DJ, Taylor CC (1994) Machine learning, neural and statistical classification. Ellis Horwood, New york
27. Phua C, Alahakoon D, Lee V (2004) Minority report in fraud detection: classification of skewed data. SIGKDD Explor 6(1):50–59
28. Prati RC, Batista GEAPA, Monard MC (2011) A survey on graphical methods for classification predictive performance evaluation. IEEE Trans Knowl Data Eng 23(11):1601–1618
29. Prati RC, Batista GEAPA, Silva DF (2013) Paper website. http://sites.labic.icmc.usp.br/ClassImbalanceRevisited/
30. Provost FJ, Fawcett T, Kohavi R (1998) The case against accuracy estimation for comparing induction algorithms. In: Shavlik JW (ed) International conference on machine learning. Morgan Kaufmann, Los Altos, CA, pp 445–453
31. Quinlan JR (1993) C4.5: programs for machine learning. Morgan Kaufmann Publishers Inc, Los Altos, CA
32. Wallace B, Small K, Brodley C, Trikalinos T (2011) Class imbalance, redux. In: IEEE international conference on data mining, pp 754–763
33. Wang X, Matwin S, Japkowicz N, Liu X (2013) Cost-sensitive boosting algorithms for imbalanced multi-instance datasets. In: Zaïane OR, Zilles S (eds) Canadian conference on artificial intelligence, vol 7884 of lecture notes in computer science. Springer, Berlin, pp 174–186
34. Weiss GM (2004) Mining with rarity: a unifying framework. SIGKDD Explor 6(1):7–19
35. Weiss GM, McCarthy K, Zabar B (2007) Cost-sensitive learning vs. sampling: which is best for handling unbalanced classes with unequal error costs? In: IEEE international conference on data mining, pp 35–41
36. Weiss GM, Provost F (2003) Learning when training data are costly: the effect of class distribution on tree induction. J Artif Intell Res 19:315–354
37. Wu G, Chang EY (2003) Class-boundary alignment for imbalanced dataset learning. In: Workshop on learning from imbalanced Datasets in international conference on machine learning

**Ronaldo C. Prati** received a B.Sc. degree in Computer Science in 2001 and a M.Sc. and a Ph.D. degrees in Computer Science and Computational Mathematics in 2003 and 2006, respectively, from University of São Paulo, campus of São Carlos, São Paulo, Brazil. Since 2009, he is an assistant professor of Computer Science at Centro de Matemática, Computação e Cognição at the Universidade Federal do ABC (UFABC), Santo André, São Paulo, Brazil. His research interests include machine learning, data and text mining, and their applications.

**Gustavo E. A. P. A. Batista** was born in São Paulo, Brazil, in 1973. He received the bachelor's degree in Computer Science from the Universidade Estadual Paulista, Rio Claro, Brazil in 1994, and M.Sc. and Ph.D. degrees in Computer Science and Computational Mathematics from Universidade de São Paulo in São Carlos, São Paulo, Brazil in 1997 and 2003, respectively. In 2007 he joined the Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, Brazil, as assistant professor. In 2010 and 2011, he was a visiting researcher at University of California, Riverside. Dr. Batista is member of Institute of Electrical and Electronics Engineers, Society for Industrial and Applied Mathematics, Association for Computing Machinery and the Brazilian Computer Society. His research interests include machine learning, data mining and time series processing with applications in public health, agriculture and environment.

**Diego F. Silva** received the B.Sc. degree in Computer Science and M.Sc. degree in Computer Science and Computational Mathematics from University of São Paulo, Brazil, in 2011 and 2014, respectively. He is currently pursuing a Ph.D. at University of São Paulo. His main research interests are machine learning, data mining and their applications.